

AVANCE MISIÓN 2:

Seguridad – IAM y Seguridad en Contenedores

1. ¿Tu proyecto utilizará recursos de AWS que requieren permisos IAM personalizados? (Recuerda que en AWS Academy no tienes permisos para IAM)

No utilizaremos permisos IAM personalizados porque la cuenta de AWS Academy tiene restricciones: no permite crear, modificar ni eliminar políticas IAM ni roles personalizados. Solo podemos usar los roles y permisos preconfigurados que ofrece la academia. Sin embargo, nos enfocaremos en buenas prácticas de seguridad a nivel de aplicación: validación, sanitización, y protección contra inyección, sin depender de IAM granular.

Refiriéndonos a la teoría y buenas prácticas, para nuestro proyecto idealmente sí se requerirían permisos IAM personalizados y bien granulados. En una arquitectura real de producción, necesitaríamos crear políticas IAM específicas para cada componente:

Lo que idealmente se debería hacer es:

- **Servicio de Pedidos (.NET):** Permisos específicos para leer/escribir en DynamoDB (tabla de pedidos) y acceso a DynamoDB Streams.
- **Servicio de Administración (Kotlin):** Permisos para actualizar estados en DynamoDB y generar eventos.
- **Servicio de Notificaciones (Python):** Permisos para consumir mensajes de SQS y enviar correos con SES.
- **Lambda Function:** Permisos para leer de DynamoDB Streams, escribir en SQS, y acceder a KMS.
- **Roles IRSA (IAM Roles for Service Accounts):** Para que cada pod en EKS tenga exactamente los permisos que necesita, sin credenciales explícitas.

Todo esto se implementaría con el principio de **mínimo privilegio**, donde cada servicio solo tiene acceso a los recursos que necesita, nada más.

2. ¿Qué mecanismo utilizarás para pasar secretos y credenciales a los contenedores? (Kubernetes Secrets, variables de entorno, archivos .env, AWS Secrets Manager, otros.)

Utilizaremos AWS Secrets Manager + Secrets Store CSI Driver, K8s Secrets y ConfigMaps

- **AWS Secrets Manager + Secrets Store CSI Driver**

Los secretos y credenciales sensibles se almacenarán en AWS Secrets Manager y serán inyectados en los pods a través del Secrets Store CSI Driver, que es la integración nativa entre EKS y Secrets Manager.

Flujo:

1. Todos los secretos (credenciales de BD, claves API, tokens SES) se almacenan en AWS Secrets Manager.
2. El Secrets Store CSI Driver monta estos secretos como volúmenes virtuales en cada pod en tiempo de ejecución.
3. Los contenedores leen los secretos directamente desde el filesystem del pod (ruta: `/mnt/secrets/`).
4. Los secretos nunca aparecen en variables de entorno (más seguro contra exposure en logs).

Ventajas:

- Secretos cifrados en reposo con KMS.
- No hay credenciales en el código ni en archivos `.env`.
- Rotación centralizada: cambiar un secreto en Secrets Manager se refleja automáticamente en todos los pods en el siguiente ciclo de montaje.

- ConfigMaps y archivos `.env`

Configuraciones públicas (URLs de servicios, puertos, ambiente) irán en **ConfigMaps** y para ambientes de preproducción, o pruebas locales, usaremos los archivos `.env`

<u>Tipo de Dato</u>	<u>Almacenamiento</u>	<u>Método de Inyección</u>
Credenciales BD, API Keys, tokens	AWS Secrets Manager	Secrets Store CSI Driver (volumen en <code>/mnt/secrets/</code>)
Configuración pública, URLs, puertos	ConfigMaps	Variables de entorno en Deployment
<code>.env</code> locales (solo desarrollo)	Archivos en repositorio	<code>.gitignore</code> , nunca en git

3. ¿Tienes claro cómo debes configurar los Grupos de Seguridad?

Sí, los recursos se van a aislar bajo el principio de menor privilegio de esta manera:

Grupo de Seguridad del Load Balancer: Es el único componente expuesto públicamente. Estará ubicado en las subredes públicas y contará con un AWS WAF (Web Application Firewall) acoplado para mitigar ataques. Sólo aceptará tráfico externo desde internet hacia los endpoints públicos de la aplicación (usuario y administración)

Grupo de Seguridad de los nodos desplegados en EKS: Para proteger las instancias EC2 donde corren los Pods (los 3 servicios backend) se usarán subredes privadas, nunca tendrán IP pública. Solo se permite tráfico entrando que provenga del grupo de seguridad del ALB. En lugar de abrir el tráfico hacia el internet público para consumir servicios como el DynamoDB, el tráfico se redirige internamente mediante las subredes privadas de la VPC.

4. ¿Consideras necesario abrir puertos para acceder a las instancias EC2 como el TCP 22, o puedes considerar alternativas como Session Manager?

No es necesario abrir el puerto TCP 22 para acceso administrativo; la solución propuesta, contempla el uso de AWS Fargate, como servicio serverless, por lo que no existiría acceso SSH directo a las instancias. Con AWS Fargate se simplifica la administración de la infraestructura, solo definimos las imágenes Docker, CPU, memoria, red, variables de entorno y políticas IAM, AWS ejecuta el contenedor automáticamente y bajo demanda, es decir, pago por uso (CPU/RAM consumida).

Para nodos o instancias EC2 (un clúster EKS híbrido o autogestionado), se priorizaría el uso de AWS Systems Manager Session Manager como mecanismo de acceso seguro y auditado, evitando exponer el puerto 22 a Internet y reduciendo la superficie de ataque (disminuir la cantidad de puntos por donde un atacante podría intentar entrar/comprometer el sistema como puertos abiertos, servicios expuestos, credenciales, APIs públicas, servidores, protocolos inseguros o accesos administrativos).

Con lo anterior, se logra enfoque de seguridad, reducción de un ataque, uso de servicios administrados y buenas prácticas cloud-native

5. En caso de que requieras almacenar datos en S3, ¿utilizarías cifrado en reposo con KMS?

Nuestra solución requiere de S3, pero únicamente para alojar allí nuestra aplicación frontend. Por lo tanto, no usaremos cifrado en reposo con KMS.

Cabe aclarar que si S3 fuera a ser utilizado en algún momento, y de que la información allí almacenada fuera delicada como información personal o financiera, definitivamente optaríamos por usar KMS tanto en reposo como en tránsito.

6. Para los contenedores, ¿requieres limitar la cantidad de recursos (CPU, memoria, disco) utilizando cgroups?

Si, se tiene contemplado limitar los recursos de los contenedores con cgroups, estos parámetros se administran a través de EKS y Docker al momento de escribir los manifiestos en los bloques *resources.limits* y *resources.requests*, esto con el fin de evitar que el servicio de pedidos (que es el candidato principal a recibir picos de carga) experimente alta demanda, evitar que consuma toda la RAM o la CPU de la instancia.

LIMITACIÓN DE RECURSOS

Contenedor/servicio	Tecnología	CPU Request/limit	Memoria Request/limit
Servicio de pedidos	.NET	100m/250m	128Mi/ 256Mi
Servicio de administracion	Kotlin	100m/250m	192Mi/ 312Mi
Servicio de notificaciones	Python	50m/150m	64Mi / 128Mi
Frontend Web	React + Nginx	50m/100m	32Mi / 64Mi

Nota: 100m (milicores) equivale al 10% de un núcleo de cpu, un límite de 250m equivale a que un contenedor no podrá usar más del 25% de un núcleo.

Una instancia t3.small ofrece 2 vCPUs y 2GiB de memoria RAM, el criterio de éxito exige que **cada servicio tenga 2 réplicas** para balanceo de carga. Sumando los *Requests* de las dos réplicas de todos los servicios, consumirás aproximadamente 600m de CPU y 832Mi de RAM en total. Esto deja espacio libre para los agentes internos de Kubernetes (como el de CloudWatch o Secrets Manager)

7. ¿Qué medidas aplicarás para reducir el riesgo dentro de los contenedores? o ¿Escanearás imágenes con Trivy, Docker Scout u otra herramienta? o ¿Aplicarás políticas de seguridad como AppArmor, seccomp, capabilities? o ¿Vas a firmar las imágenes que vas a utilizar con una herramienta como cosign?

Para minimizar el riesgo lo ideal es una estrategia por capas (Defense in Depth).

Para el Runtime limitar todas las capacidades de los contenedores y dejar solo las mínimas necesarias mediante el *securityContext*. Seccomp para filtrar las llamadas que el contenedor puede hacer al kernel y AppArmor para definir lo que cada contenedor puede hacer en el sistema de archivos.

Para el escaneo de imagenes usaremos SonarQube para analisis del código fuente y sumaremos Trivy para complementar la seguridad del código, bloqueando el pipeline si la imagen tiene fallos graves.

Por último, habilitar el firmado de imágenes para evitar que se alteren las mismas.

8. ¿Consideras oportuno utilizar RBAC?

Si, es oportuno, el proyecto contempla separar los ambientes en namespaces (Dev y Prod). Con RBAC se asegurará que no se realicen cambios en Prod. mientras se realiza el desarrollo. Además nos permite cumplir con el principio del mínimo privilegio.

Diagnóstico y Depuración

9. ¿Cómo piensas verificar que los pods están funcionando correctamente en desarrollo y en producción?

En desarrollo lo ideal es realizar un diagnóstico inicial mediante la correlación de comandos de `kubectl` (`get pods`, `describe` para analizar eventos de fallos como `ImagePullBackOff` o `Pending`, y `logs`).

En producción se usará la verificación continua mediante la configuración de *Liveness* y *Readiness Probes* en los manifiestos YAML para gestionar la salud del runtime y mitigar errores, simultáneamente se centralizará la observabilidad recolectando métricas e ingiriendo registros estructurados en JSON hacia Amazon CloudWatch Logs.

10. ¿Tienes alguna estrategia para definir los logs?

Se implementará una estrategia de logs estructurada y centralizada, con el objetivo de facilitar el diagnóstico y la observabilidad, los logs se consultarán en herramientas como CloudWatch Logs o Logs Insights

Todos los servicios generarán logs en formato JSON estructurado para facilitar su indexación y búsqueda. Cada registro incluirá:

- Timestamp exacto del evento -> timestamp
- Nombre del microservicio/Componente -> service
- Nivel de severidad (INFO, WARN, ERROR, DEBUG) -> severity
- Identificador único de solicitud (Correlation ID / UUID) -> requestId
- Id del usuario -> userId
- Endpoint o acción ejecutada -> endpoint-action
- Mensaje -> message
- Descripción detallada -> description

Ejemplo json:

```
{
  "timestamp": "2026-06-02T18:00:21Z",
  "service": "orders-service",
  "severity": "ERROR",
  "requestId": "8c5e-43ff-a22",
  "userId": "123456",
  "endpoint-action": "/orders",
  "message": "Database connection timeout",
  "description": "System.Data.SqlClient.SqlException (0x80131904): Connection Timeout Expired"
}
```

La estrategia contempla:

- Logs de aplicación → comportamiento del negocio
- Logs de infraestructura → Kubernetes, pods, eventos del clúster
- Logs de auditoría → cambios importantes o eventos críticos

11. ¿Qué comando o herramientas usarás para obtener logs o eventos en caso de falla? Ej: kubectl logs, kubectl describe, CloudWatch Logs, Logs Insights

Para el diagnóstico en kubernetes, se usarán los siguientes comandos:

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl logs -f <pod-name>
kubectl get events --sort-by=.metadata.creationTimestamp
```

Uso esperado:

- kubectl logs → revisar errores de aplicación
- kubectl describe → inspeccionar eventos como CrashLoopBackOff, OOMKilled, ImagePullBackOff
- kubectl get events → identificar fallos del clúster

Observabilidad centralizada:

- CloudWatch Logs → almacenamiento centralizado de logs
- Logs Insights → consultas avanzadas y búsqueda de errores
- Grafana Explore → análisis visual y correlación
- Prometheus → correlación de métricas con incidentes

Flujo común:

1. Alerta generada (CloudWatch Alarms)
 - a. Saber que existe un problema
2. Identificar servicio/componente afectado (Grafana Dashboards / kubectl get pods)
3. Consultar métricas (Prometheus + Grafana de forma gráfica y más amigable)
 - a. Ver CPU, memoria, latencia, errores
4. Revisar logs centralizados (CloudWatch Logs / Logs Insights)
 - a. Encontrar causa raíz
5. Verificar eventos del clúster y estado del pod (kubectl describe / kubectl get events)
 - a. Detectar CrashLoopBackOff, OOMKilled, etc
6. Aplicar corrección y validar recuperación (kubectl rollout restart, kubectl logs, Grafana + Probes)
 - a. Reiniciar, redeploy, rollback
 - b. Confirmar que volvió a la normalidad

Este flujo permite pasar de una alerta automática a un diagnóstico guiado, correlacionando métricas, logs y eventos del clúster para reducir el tiempo de resolución de incidentes

12. ¿Tienes un plan para simular errores controlados en el entorno (stress test, contenedores caídos, pérdida de conexión)?

Se implementarán pruebas controladas en ambientes de desarrollo o staging para validar resiliencia, observabilidad y recuperación automática. Se contemplan las siguientes pruebas:

- **Stress Testing:** Simular consumo elevado de CPU o memoria, con el objetivo de validar límites de recursos, verificar auto recuperación y observar el comportamiento bajo carga
 - `stress-ng --cpu 2 --vm 1 --vm-bytes 512M`
- **Fallos de contenedores:** Simular la eliminación manual de pods, imágenes defectuosas, intermitencias, procesos terminados abruptamente, con el objetivo de validar self-healing de Kubernetes y verificar probes
 - `kubectl delete pod <pod>`
- **Fallos de conectividad:** Simular caída de BD, pérdida de conectividad, fallos entre microservicios, con el objetivo de validar retry policies, validar circuit breakers, mirar el comportamiento degradado
- **Chaos Testing básico:** Simular diferentes escenarios como pods reiniciando frecuentemente, indisponibilidad parcial o fallos múltiples de forma simultánea

13. ¿Configurarás algún mecanismo para detectar errores comunes como: • Fallos de conexión a base de datos • Crash del backend • Servicios no disponibles • Problemas de permisos Monitoreo – Tradicional, Serverless y Configuración

Se configurarán mecanismos preventivos y reactivos para detectar automáticamente fallos frecuentes

- **Fallos de conexión a base de datos:** logs estructurados, timeouts configurados, métricas de errores, alertas automáticas
- **Crash del backend:** detección mediante liveness probes, reinicios automáticos, eventos Kubernetes, métricas de reinicios
 - Ejemplo:
livenessProbe:
httpGet:
path: /health
port: 8080
- **Servicios no disponibles:** detección mediante readiness probes, monitoreo de endpoints y métricas de disponibilidad, con el objetivo de evitar enviar tráfico a pods dañados
- **Problemas de permisos:** aquí se puede detectar por medio de logs de autorización, fallos IAM , errores de acceso a recursos AWS

- **Sistema de alertas:** Grafana Alerts + CloudWatch Alarms

14. ¿Qué tipo de monitoreo implementarás en tu proyecto?

- **Tradicional (EC2, CloudWatch básico)**
- **Serverless (Prometheus, Grafana, métricas de pods)**
- **Monitoreo de configuración (AWS Config)**
- **Combinación de varios**

La mejor opción para nuestro proyecto es una combinación de varios, específicamente CloudWatch + Prometheus + Grafana:

- **CloudWatch** para todo lo que viene de los servicios administrados por AWS (DynamoDB, SQS, SES, Lambda) sin configuración adicional — AWS lo provee de forma nativa.
- **Prometheus + Grafana** para los pods en EKS, donde necesitas métricas a nivel de aplicación (latencia de endpoints, errores HTTP, uso de memoria por pod) que CloudWatch no expone de forma granular. Esta combinación es el estándar de la industria para arquitecturas en Kubernetes y es lo que mejor se alinea con EKS.

15. ¿Vas a recolectar métricas personalizadas del backend o frontend?

Sí, con foco en el backend. Las métricas más relevantes son:

- **En el servicio de pedidos (.NET):** tiempo de respuesta por endpoint, tasa de errores 4xx/5xx, y número de pedidos creados por minuto.
- **En el servicio de administración (Kotlin):** cantidad de cambios de estado procesados y latencia de actualización en DynamoDB.
- **En el servicio de notificaciones (Python):** número de mensajes consumidos desde SQS, tiempo entre cambio de estado y envío del email, y tasa de fallos de envío en SES.
- El frontend en S3/CloudFront no requiere métricas personalizadas para esta fase — CloudFront ya expone métricas nativas de requests y errores.

16. ¿Vas a crear dashboards en herramientas como Grafana para visualizar el estado del sistema?

Sí. Con Prometheus recolectando las métricas de los pods, Grafana se conecta como fuente de datos y permite visualizar el estado del sistema en tiempo real. Para esta fase un dashboard básico con los siguientes paneles es suficiente y profesional:

- Estado general de los pods (réplicas activas vs deseadas)
- Latencia promedio por microservici
- Tasa de errores HTTP

- Longitud de la cola SQS (indicador clave del flujo de notificaciones)
- Número de pedidos creados en las últimas 24 horas.

17. ¿Vas a incluir liveness y readiness probes para mejorar la observabilidad de los pods?

Sí, y es uno de los puntos más importantes para un despliegue serio en EKS. Sin ellos, Kubernetes no puede distinguir un pod que arrancó pero está roto de uno que funciona correctamente.

- **Liveness probe:** le dice a Kubernetes si el pod está vivo. Si falla, lo reinicia automáticamente. Se implementa como un HTTP GET al endpoint de health del servicio, por ejemplo `GET /health`.
- **Readiness probe:** le dice a Kubernetes si el pod está listo para recibir tráfico. Si falla, el pod se saca del load balancer hasta que se recupere

Para los tres servicios la implementación es idéntica en concepto, solo cambia el puerto y el path según el framework (.NET, Spring Boot, FastAPI).

18. ¿Utilizarás AWS Config para validar que la infraestructura cumpla con ciertas políticas?

Consideramos que sí. Es importante, para este proyecto al ser enfocado en mediana empresa, que los costos no superen cierto umbral para no preocupar a nuestros clientes ni mucho menos nuestra operación, también es importante para nosotros cumplir con normas de almacenamiento de información personal y para esto AWS config será útil al ser un auditor de cada recurso lanzado en AWS. Consideramos que su integración con SNS es muy importante ya que por medio de la misma los administradores del sistema serán alertados si AWS config encuentra alguna configuración no deseada en cualquier recurso: base de datos, cantidad de instancias de kubernetes, configuraciones de seguridad riesgosas, etc.

Consideraciones generales

19. ¿Qué medidas vas a aplicar desde la Misión 2 para hacer que tu proyecto sea más seguro, observable y fácil de mantener?

las medidas de seguridad, observabilidad y mantenibilidad implementadas en nuestro proyecto serán:

SEGURIDAD

- Mantener todo usuario y aplicación del sistema con un ROL o Cuenta de IAM con los mínimos permisos requeridos para su correcto funcionamiento
 - Nuestras aplicaciones deberán tener un rol que les permita usar DynamoDB, Lambda, SQS y nada más. Inclusive estos permisos se van a discriminar entre permisos de lectura y escritura. Esto evita que nuestros servidores

puedan ser punto de ataque a nuestra infraestructura y/o información almacenada.

- Información de base de datos cifrada de manera estática y en tránsito
- La configuración de secretos y llaves será implementada fuera de las imágenes docker
- Se configurará para cada aplicación un usuario en particular
- Toda imagen generada por nuestro pipeline de despliegue será escaneada con Trivy en búsqueda de vulnerabilidades
- Configuración de Cgroups para mantener el uso de recursos y memoria en un nivel que no sea peligroso para nuestra operación
- Implementaremos Seccomp para reducir el alcance del impacto en un contenedor comprometido por un ataque de seguridad
- El uso de RBAC también es clave para reducir los permisos de un contenedor sobre nuestra plataforma

OBSERVABILIDAD

- Logs estructurados correctamente para el seguimiento y debug de errores
- Tracing de requests
- Crearemos dashboards personalizados con las métricas:
 - uso de CPU de instancias
 - cantidad de requests por minuto
 - errores 5XX, 4XX por minuto
 - success request por minuto
 - tiempo en que se tomó cada request

MANTENIBILIDAD

- AWS config será un servicio clave de AWS que nos ayudará a mantener las configuraciones de la nube de nuestra aplicación siempre iguales. Puertos habilitados desde grupos de seguridad, usar o no instancias dedicadas, apagar o encender alarmas, etc será constantemente monitoreado por AWS config.
- El utilizar servicios de AWS serverless también nos ayuda en el aspecto de mantenibilidad al ahorrarnos la configuración y mantenimiento de los recursos en la nube. El uso de lambda, dynamo, SES, SQS, SNS y Fargate en nuestra aplicación es clave para este aspecto al ser servicios completamente administrados por aws

20. ¿Qué conceptos decidiste no implementar? ¿Cuál fue la razón?

Consideramos que el uso de imágenes firmadas es un mecanismo que no implementaremos ya que las imágenes generadas de nuestra aplicación no estarán abiertas al público y serán accesibles solo por y para nosotros.

21. ¿Qué impacto tendría omitir la configuración de monitoreo y seguridad en una posible versión productiva de tu solución?

El omitir seguridad y monitoreo en una versión productiva de nuestro servicio sería bastante peligroso dado que estamos hablando del lanzamiento de un MVP.

El hecho de lanzar la primera versión de nuestra aplicación sin ningún tipo de observabilidad sobre lo que pasa en nuestro sistema reduce nuestra capacidad de anticipar

errores y nos daremos cuenta de ellos en el momento en que nuestros clientes nos informen de ellos; degradando así la experiencia de nuestros usuarios.

Ahora, si hablamos de seguridad, sería sumamente peligroso ya que estaríamos exponiendo nuestro sistema a ataques y robo de información que podría ser utilizada para estafas y malas intenciones.

22. ¿Cómo verificarás y validarás las decisiones tomadas sobre seguridad, observabilidad y diagnóstico en la presentación final?

- Validación de Seguridad (Demostración de resiliencia y mínimo privilegio)
 - Reportes de Escaneo de Vulnerabilidades: Mostraremos el artefacto o reporte generado por el pipeline de CI/CD donde Trivy analiza las imágenes Docker, demostrando que el despliegue se bloquea si existen vulnerabilidades críticas.
 - Prueba de Denegación de Permisos (Principio de Mínimo Privilegio): Simularemos un acceso interactivo a uno de nuestros contenedores en ejecución (ej. en AWS Fargate) e intentaremos ejecutar un comando destructivo o acceder a un recurso no autorizado en AWS (como listar buckets de S3 o intentar escribir en una tabla de DynamoDB para la cual el rol de IAM solo tiene permisos de lectura). Mostraremos en vivo el error de **AccessDenied**.
 - Evidencia de Configuración: Mostraremos brevemente los archivos de configuración (**securityContext** de Kubernetes/ECS) donde se evidencia la restricción de recursos mediante Cgroups y el perfil de Seccomp activo.
- Validación de Observabilidad (Transparencia del sistema)
 - Recorrido del Dashboard en Vivo: Durante la demo, realizaremos peticiones a la aplicación web (simulando tráfico real) y mostraremos en tiempo real cómo impacta nuestro Dashboard personalizado. Validaremos que se grafiquen correctamente el uso de CPU, las peticiones por minuto y los tiempos de respuesta.
 - Inyección y Rastreo de un Error: Provocaremos intencionalmente un error (por ejemplo, enviando un parámetro corrupto para forzar un error 4XX o 5XX) para demostrar cómo los logs estructurados y el Tracing de la request nos permiten aislar, identificar y diagnosticar la causa raíz del fallo en cuestión de segundos, sin tener que adivinar.
- Validación de Diagnóstico y Cumplimiento (Mantenibilidad)
 - Auditoría de AWS Config: Mostraremos el panel de AWS Config en un estado "Compliant" (Cumplido), validando que las reglas automáticas que definimos (como el bloqueo de puertos públicos en los Security Groups o el cifrado de

datos) están siendo monitoreadas y respetadas por la infraestructura en la nube.